SYNTH-99: Best Practice Summary

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/19/2026

Overview

This notebook consolidates every actionable best practice from the SYNTH series (notebooks 01-06) into a single definitive reference. Each practice specifies the exact setting or value to use, its priority level, and category. Use this as a checklist when deploying or auditing Dynatrace synthetic monitoring.

Table of Contents

1. Monitor Selection
2. HTTP Monitor Configuration
3. Browser Monitor Configuration
4. Network Monitor Configuration
5. Scheduling and Frequency
6. Location Strategy
7. Private Locations and ActiveGate
8. Validation and Assertions
9. Authentication and Security
10. Performance Thresholds
11. Alerting Configuration
12. SLOs and Error Budgets
13. Dashboard and Reporting
14. Operational Hygiene

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring enabled
- ✅ Completed SYNTH-01 through SYNTH-06 (or using as a standalone reference)
- ✅ DQL query permissions (viewer role minimum)

1. Monitor Selection

Choose the right monitor type for each use case.

Practice	Recommended Setting/Value	Priority
Use **HTTP monitors** for API health checks and REST endpoints	Monitor type: HTTP single request	Critical
Use **HTTP multi-step** for auth-token-then-API workflows	Monitor type: HTTP multi-step with variable extraction	Critical
Use **Single-URL browser monitors** for homepage/landing page availability	Monitor type: Browser single-URL	Critical
Use **Browser clickpath monitors** for multi-page user journeys (login, checkout)	Monitor type: Browser clickpath	Critical
Use **ICMP monitors** for host reachability and latency baseline	Monitor type: Network availability - ICMP	Recommended
Use **DNS monitors** for name resolution validation	Monitor type: Network availability - DNS	Recommended
Use **TCP port monitors** for service port connectivity (databases, caches)	Monitor type: Network availability - TCP	Recommended
Layer monitors: DNS + ICMP + TCP + HTTP for full-stack coverage of critical services	Deploy one monitor per OSI layer for each critical path	Recommended
Prefer HTTP monitors over browser monitors when JavaScript rendering is not needed	Saves cost and runs faster (< 1s vs 3-30s)	Recommended

Selecting Monitors in Bulk (SaaS 1.344)

Monitor *selection* is a separate concern from monitor *type*, so it sits outside the table above rather than as a row in it. **SaaS 1.344** (released 07/27/2026, **staged tenant rollout from 07/29/2026**) lets a single monitor selection combine **AND and OR** operators, instead of requiring one selection per tag:

Selector form	Meaning
`tag(tag1),tag(tag2)`	logical **AND** — the monitor must carry both tags
`tag(tag1,tag2)`	logical **OR** — the monitor carries either tag

So `tag(env:prod),tag(team:payments)` selects the production payments monitors in one expression. Before 1.344 this needed separate selections per tag, then a union downstream — which is why bulk operations and tag-scoped automation tended to fragment.

Verify 1.344 has reached your tenant before relying on the combined form; until it does, the per-tag selection approach remains the working path, and it keeps working afterwards.

> **Sources:** Dynatrace SaaS release notes 1.344 (DT docs) — "`tag(tag1),tag(tag2)` is the logical AND", "`tag(tag1,tag2)` is the logical OR".

2. HTTP Monitor Configuration

Practice	Recommended Setting/Value	Priority
Set a per-request performance threshold where response time matters	*Generate a problem and send an alert on performance threshold violations* on the request	Recommended
Set `Content-Type` and `Accept` headers on every request	`Content-Type: application/json`, `Accept: application/json`	Critical
Add User-Agent header for identification	`User-Agent: Dynatrace Synthetic`	Recommended
Keep certificate validation on, and alert before expiry	Invalid certificates fail the monitor by default (leave *Accept any SSL certificate* off); turn on the expiry problem (n ≤ 100 days)	Critical

Practice	Recommended Setting/Value	Priority
Validate API responses with a text/regex rule, or with `api.fail()` in a post-execution script for structured checks (JSON-path assertions are not a rule type)	Regex rule: `"status":\s*"success"` (evaluated over the first 50 KB of the body)	Critical
Pass values between requests with a post-execution script	`api.setValue("token", JSON.parse(response.getResponseBody()).access_token)`	Critical
Reference stored values in later requests as `{variable_name}` — single braces, no `$`	URL: `https://api.example.com/users/{userId}`	Critical
Validate HTTP status codes	Response status code validation is a rule type, evaluated first	Critical
Add content-present assertion for positive match	Validation: contains `"status": "ok"`	Recommended
Add content-absent assertion to catch error states	Validation: does not contain `"error"`	Recommended

> **Sources:** Create and configure an HTTP monitor (DT docs), Pre- and post-execution scripting for HTTP monitors (DT docs) — *"You can also apply the value of a variable previously set using api.setValue() in subsequent HTTP monitor configuration fields using the {variable_name} convention."*

3. Browser Monitor Configuration

Practice	Recommended Setting/Value	Priority
Set viewport to match target audience device	Desktop: **1920x1080**, Laptop: **1366x768**, Tablet: **768x1024**, Mobile: **375x667**	Critical

Practice	Recommended Setting/Value	Priority
Use CSS selectors as the primary element selector type	Selector: `#login-btn` or `[data-testid='login']`	Critical
Use `data-testid` attributes for automation-stable selectors	Selector: `[data-testid='submit']`	Recommended
Avoid XPath selectors unless DOM structure requires it	Use CSS first; XPath only for complex structures	Recommended
Add content validation on every clickpath step	Validate expected text/element exists after each action	Critical
Record clickpaths with the Dynatrace Synthetic Recorder Chrome extension	Recorder captures reliable events and selectors	Recommended
Enable automatic screenshots on success and failure	Screenshots: enabled (default)	Critical
Keep clickpath steps minimal (only critical path)	Fewer steps = faster execution and fewer false positives	Recommended
Do NOT rely on session persistence between executions	Cookies are maintained within a single execution only	Critical

4. Network Monitor Configuration

NAM monitors run only on private locations. The values below are starting points from community practice unless quoted.

Practice	Recommended Setting/Value	Priority
Send several ICMP packets per request	Number of packets: 3–5 (range 1–10, default 1)	Recommended
Name the DNS server and record types explicitly	DNS server + `A` (and `AAAA` where you serve IPv6)	Critical
Monitor TCP ports for every critical service: 443 (HTTPS), 5432 (PostgreSQL), 6379 (Redis), 3306 (MySQL), 27017 (MongoDB)	Port ranges per request	Critical
Layer DNS, ICMP and TCP checks on critical infrastructure	One monitor per type per target, so each raises its own problem	Recommended

Practice	Recommended Setting/Value	Priority
Set the step success-rate constraint deliberately	Default ≥ 80%; 100 for single-target requests (the docs suggest it)	Recommended

5. Scheduling and Frequency

Practice	Recommended Setting/Value	Priority
Run HTTP monitors at **1-5 minute** intervals for critical APIs	Frequency: **1 min** (critical), **5 min** (standard)	Critical
Run browser monitors at **5-15 minute** intervals	Frequency: **5 min** (critical), **15 min** (standard)	Critical
Run network monitors at **1-5 minute** intervals	Frequency: **1 min** (infrastructure), **5 min** (standard)	Recommended
Use **5-minute** frequency as the default for all non-critical monitors	Frequency: **5 minutes**	Recommended
Never run browser monitors more frequently than every 5 minutes	Minimum interval: **5 min** (browser)	Critical
Match frequency to SLA detection requirements: 1 min frequency detects outages within 2-3 min	Frequency = desired detection time / consecutive-failure count	Recommended

6. Location Strategy

Practice	Recommended Setting/Value	Priority
Assign a minimum of **3 public locations** per external-facing monitor	Locations: 3+ geographically distributed	Critical
Select locations that match your user base geography	Choose regions where real users access the app	Critical
Use public locations for external-facing applications only	Location type: Public (Dynatrace-hosted)	Critical
Use private locations for internal applications, VPN-only apps, and pre-production	Location type: Private (ActiveGate)	Critical

Practice	Recommended Setting/Value	Priority
Require failures from **2+ locations** before raising an outage alert	Outage detection: multi-location confirmation	Critical
Include at least one location per major region (Americas, EMEA, APAC) for global apps	1+ location per continent where users exist	Recommended

7. Private Locations and ActiveGate

Practice	Recommended Setting/Value	Priority
Deploy ActiveGate with `--enable-synthetic` capability	Capability: **synthetic**	Critical
Size each synthetic ActiveGate from the sizing guide (XS/S/M/L by executions per hour), with at least **8 GB RAM and 25 GB free disk**	XS minimum is 2 vCPU / 4 GB / 20 GB; docs recommend 8 GB RAM / 25 GB disk; resizing means reinstalling (SYNTH-04)	Critical
Deploy **2+ ActiveGates** per private location for high availability	Nodes per location: **2+**	Critical
Distribute ActiveGates across availability zones	AZ distribution: one AG per AZ minimum	Recommended
Let the ActiveGate installer provide the browser, and keep it current with the per-location **Enable Chrome(-ium) auto-update** switch (Linux; on by default) — not by changing ActiveGate auto-update	Browser updates during ActiveGate and Synthetic engine updates; Windows always updates it; supported window is two versions behind the latest (SYNTH-04)	Critical
Keep every ActiveGate in a private location on the same version	One ActiveGate version per location	Recommended
Monitor ActiveGate CPU < 80%, memory < 80%, disk < 80%	Alert thresholds: **80%** sustained for CPU/memory/disk	Critical
Monitor ActiveGate execution queue depth	Alert threshold: queue > **100** pending executions	Recommended
Deploy containerized private locations from the UI-generated `synthetic.yaml` template, not from DynaKube	Template: Synthetic → Private locations → Download synthetic.yaml (plus the metric-adapter template)	Recommended
Ensure outbound-only connectivity (no inbound firewall rules required)	Network: HTTPS outbound to Dynatrace cluster only	Critical

> **Sources:** Requirements for private Synthetic locations (DT docs) — *"On Windows, the ActiveGate installer package includes the Chrome for Testing browser used to run browser monitors."*; Containerized private Synthetic locations on Kubernetes (DT docs) — *"Select Download synthetic.yaml. This is the location template file."*; Manage private Synthetic locations (DT docs) — *"the browser autoupdate is turned on by default for locations with Linux-based ActiveGates"* and *"We strongly recommend updating all ActiveGates per location to the same version."*

8. Validation and Assertions

Practice	Recommended Setting/Value	Priority
Add at least one content validation rule to every monitor	Validation: text-present or regex rule (HTTP), or a post-execution `api.fail()` check	Critical
Validate positive content ("Welcome", "status: ok") rather than only checking for absence of errors	Validation type: **Text Present**	Critical
Use regex validation for dynamic content (e.g., order numbers)	Regex: `Order #\d{6}`	Recommended
Validate JSON responses with a text/regex rule or a post-execution script — JSON-path assertions are not a rule type	`var d = JSON.parse(response.getResponseBody()); if (d.status !== "success") api.fail("status=" + d.status);`	Critical
Verify element presence in browser monitors for SPA pages	Selector: `#success-message` must exist	Recommended
Enable HTTP status code validation (fail on 4xx/5xx) on all monitors	Default: fail on status 400-599	Critical
Add content-absent checks for known error strings	Validation: does not contain `"error"`, `"exception"`	Recommended

> **Sources:** Create and configure an HTTP monitor (DT docs) — *"HTTP status code validation is the most important, followed by text and regular expression validation, and finally by SSL certificate expiry validation."*

9. Authentication and Security

Practice	Recommended Setting/Value	Priority
Store all credentials in the Dynatrace Credential Vault	Path: Credential Vault	Critical
Reference credentials as `{CREDENTIALS_VAULT-<id>|username}`, `|password}` or `|token}`	Never hardcode tokens or passwords in monitor config	Critical
Use OAuth2 client_credentials flow for API authentication	Step 1: POST to token endpoint, Step 2: Bearer token in header	Recommended
Use Bearer token auth (not Basic auth) for modern APIs	Header: `Authorization: Bearer {token}`	Recommended
Use client certificates (mTLS) for high-security API endpoints	Credential type: certificate in vault	Optional
Set SSL certificate expiration warning at **30 days**	Alert: warning at 30 days before expiry	Critical
Set SSL certificate expiration critical alert at **14 days**	Alert: critical at 14 days before expiry	Critical
Set SSL certificate expiration emergency alert at **7 days**	Alert: emergency at 7 days before expiry	Critical
Validate SSL certificate chain, hostname match, and trusted CA on every HTTPS monitor	SSL checks: validity + chain + hostname + trust (all enabled)	Critical

> **Sources:** Create and configure an HTTP monitor (DT docs) — *"Use the format {|token} , {|username} , or {|password}".*

10. Performance Thresholds

Set definitive thresholds for synthetic response time.

Practice	Recommended Setting/Value	Priority
HTTP monitor response time target	Good: **< 2s**, Warning: **2-5s**, Critical: **> 5s**	Critical
DNS resolution time target	Good: **< 50ms**, Warning: **50-200ms**, Critical: **> 200ms**	Recommended
TCP connect time target	Good: **< 100ms**, Warning: **100-300ms**, Critical: **> 300ms**	Recommended
Time to First Byte (TTFB) target	Good: **< 500ms**, Warning: **500ms-1s**, Critical: **> 1s**	Critical
Take Web Vitals targets (LCP, FCP, CLS) from RUM, not synthetic	Synthetic browser monitors expose availability and step/total duration, not Web Vitals keys — see SYNTH-02 and the WEBRUM series	Recommended
Flag performance anomalies when max response time > **2x** the average	Deviation factor threshold: **2.0**	Recommended

11. Alerting Configuration

Practice	Recommended Setting/Value	Priority
Require **2-3 consecutive failures** before triggering an availability alert	Consecutive failures: **3** (recommended)	Critical
Require failures from **2+ locations** to confirm a global outage	Location threshold: **2**	Critical
Enable **Automatic retry on error** on browser monitors to absorb single transient failures	Retry: enabled (one extra execution after a failure)	Critical
Expect performance problems to close only after **five clean executions**	No separate auto-resolve or alert-delay setting exists	Recommended
Configure availability alerts for outage detection	Alert type: availability, trigger on consecutive failures	Critical
Configure performance alerts for degradation detection	Alert type: P95 response time > threshold	Recommended
Configure SLO-based alerts for error budget consumption	Alert type: error budget burn rate	Recommended

Practice	Recommended Setting/Value	Priority
Configure SSL certificate expiration alerts at 30/14/7 day thresholds	Three alert tiers: warning/critical/emergency	Critical
Use multi-location failure detection to classify local vs global outages	Severity: WARNING at 2 locations, CRITICAL at 3+ locations	Recommended

> **Sources:** Synthetic alerting overview (DT docs) — *"it configures the monitor to avoid false positives by making one more execution after the first one failed"* and *"no performance threshold is violated in the five most recent executions"*.

12. SLOs and Error Budgets

Practice	Recommended Setting/Value	Priority
Define an availability SLO for every critical synthetic monitor	SLO target: **99.9%** (8.76 hours downtime/year)	Critical
Define a performance SLO for P95 response time	SLO: **95%** of executions < **3000ms**	Recommended
Calculate error budget as `total_executions * (1 - SLO_target)`	Budget: 0.1% of executions for 99.9% SLO	Critical
Track error budget burn rate daily	Daily query: failures / daily budget allocation	Recommended
Use a **30-day rolling window** for SLO evaluation	SLO timeframe: **30 days**	Critical
Alert when error budget is **50% consumed** in less than half the window	Burn rate alert: 2x normal consumption rate	Recommended
Map SLA tiers to availability targets: 99.9% = 8.76h/yr, 99.5% = 43.8h/yr, 99.0% = 87.6h/yr	Set SLO target to match contractual SLA	Critical

13. Dashboard and Reporting

Practice	Recommended Setting/Value	Priority
Build a synthetic dashboard with these 6 tiles: Overall Availability (single value), Availability Trend (line chart), P95 Response Time by Monitor (bar chart), Location Map, Failures Table, SLO Status (gauge)	Dashboard layout: KPI row + trend row + detail row + table row	Recommended
Show overall availability as a single-value tile	Tile: `countIf(available) * 100 / count()` over 24h	Recommended
Show hourly availability trend as a line chart over 24 hours	Tile: availability_pct by 1h buckets	Recommended
Show P95 response time by monitor as a bar chart, sorted descending	Tile: `percentile(response_time, 95)` by monitor, top 10	Recommended
Show recent failures table with timestamp, monitor, location, error	Tile: last 20 failures, sorted by timestamp desc	Recommended
Include a week-over-week performance comparison	Query: this week avg/P95 vs last week avg/P95	Optional
Generate a daily availability report for the last 30 days	Query: daily availability_pct, sorted by day	Recommended

14. Operational Hygiene

Practice	Recommended Setting/Value	Priority
Name monitors descriptively: `[Type] Target - Purpose`	Example: `[HTTP] api.example.com/health - API Health`	Recommended
Name private locations with a consistent convention that distinguishes them from public locations	Example: `PRIVATE-DC1-US-EAST`	Recommended
Pair every synthetic monitor with a corresponding RUM configuration for the same application	Synthetic = baseline/SLA; RUM = actual user experience	Recommended
Review and disable stale monitors that no longer have valid targets	Audit: quarterly review of all enabled monitors	Recommended

Practice	Recommended Setting/Value	Priority
Use the HTTP timing breakdown (DNS, TCP connect, TLS, TTFB) to isolate root cause of slowness	Query: avg per phase by monitor from `http_step_execution` records	Critical
Check the private-location logs when tests fail	`/var/log/dynatrace/synthetic` (synthetic) and `/var/log/dynatrace/gateway` (ActiveGate)	Critical
Test network connectivity from ActiveGate to targets with `curl -v` and `nslookup` before configuring monitors	Pre-check: verify DNS resolution and HTTPS connectivity	Critical
Use `fetch dt.synthetic.events` (event.type `*_monitor_execution`, success via `result.state`) as the primary data source for DQL analysis; use `timeseries dt.synthetic.*` metrics for trends	Data source: `dt.synthetic.events` + synthetic metrics, not `bizevents`	Recommended
Set primary Grail tags on every monitor	Edit > Primary Grail tags (or `primaryGrailTags` in the API); max 10 per monitor; private locations need ActiveGate 1.331+ — nothing is inherited from locations	Recommended

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.